



Báo cáo AI và ứng dụng Nhóm 5

Upload for free download (Trường Đại học Bách khoa Hà Nội)



Scan to open on Studeersnel

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN - ĐIỆN TỬ



BÁO CÁO ĐỒ ÁN CHUYÊN NGÀNH I

Phân loại động tác tập gym sử dụng MediaPipe và Random Forest

Giảng viên hướng dẫn: Thầy Lưu Quang Trung

Mã Lớp: 754575

Sinh viên: Nguyễn Minh Phúc - 20241104E

Hà Nội, 06/2025

LỜI MỞ ĐẦU

Trong thời đại công nghệ 4.0, trí tuệ nhân tạo (AI) đang dần trở thành một phần không thể thiếu trong nhiều lĩnh vực của đời sống, đặc biệt là trong thị giác máy tính (Computer Vision) và nhận diện hành vi con người. Một trong những ứng dụng thực tiễn và hữu ích của lĩnh vực này là theo dõi và đánh giá tư thế tập luyện thể thao – giúp người dùng cải thiện kỹ thuật, hạn chế chấn thương, và nâng cao hiệu quả tập luyện.

Xuất phát từ nhu cầu đó, em đã chọn thực hiện đồ án “Nhận diện động tác tập gym sử dụng MediaPipe và Random Forest”. Dự án tập trung vào việc nhận dạng ba tư thế phổ biến: Squat, Push-up và Bicep Curl, từ dữ liệu video hoặc webcam. Qua đó, em có cơ hội ứng dụng kiến thức về xử lý dữ liệu pose, kỹ thuật tăng cường dữ liệu, huấn luyện mô hình học máy, và đánh giá hiệu quả mô hình thông qua các chỉ số khách quan.

Trong quá trình thực hiện, em đã rèn luyện thêm kỹ năng xử lý dữ liệu, lập trình Python, hiểu sâu hơn về các thuật toán học máy cơ bản, đồng thời nâng cao khả năng tự học và giải quyết vấn đề.

Em xin chân thành cảm ơn thầy Lưu Quang Trung đã tận tình hướng dẫn, tạo điều kiện và hỗ trợ em hoàn thành tốt đồ án này.

MỤC LỤC

DANH MỤC HÌNH ẢNH.....	
CHƯƠNG 1. GIỚI THIỆU.....	1
1.1 Tổng quan bài toán.....	1
1.2 Hướng giải quyết bài toán.....	1
1.3 Bố cục báo cáo.....	1
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT.....	2
2.1 Tổng quan về MediaPipe.....	2
2.2 Các kỹ thuật tăng cường dữ liệu được sử dụng.....	3
2.2.1. Kỹ thuật Jitter (thêm nhiễu nhỏ ngẫu nhiên).....	3
2.2.2. Kỹ thuật Mirror (lật trái-phải).....	4
2.3 Các phương pháp phân chia và đánh giá dữ liệu được sử dụng.....	5
2.3.1. Kỹ thuật Train/Test Split.....	5
2.3.2. Kỹ thuật K-fold Cross-Validation.....	6
2.4 Cơ sở lý thuyết mô hình Random Forest.....	6
2.4.1. Giải thuật Decision Tree.....	7
2.4.2. Random Forest.....	8
CHƯƠNG 3: HUẤN LUYỆN MÔ HÌNH VÀ ĐÁNH GIÁ KẾT QUẢ.....	10
3.1 Thu thập dữ liệu.....	10
3.2 Tăng cường dữ liệu.....	12
3.3 Phân chia dữ liệu và huấn luyện mô hình.....	13
3.4 Đánh giá kết quả.....	15
3.5 Các khó khăn gặp phải và hướng phát triển trong tương lai.....	17
KẾT LUẬN.....	18

DANH MỤC HÌNH ẢNH

Hình 1. Cấu trúc khung xương và các điểm khớp đặc trưng trong human pose của MediaPipe	3
Hình 2. Minh họa cách hoạt động của Jitter	4
Hình 3. Minh họa cách hoạt động của kỹ thuật Mirror	4
Hình 4. Dữ liệu gốc được chia thành 2 tập dữ liệu Train và Test	5
Hình 5. Cách kỹ thuật K-fold Cross-Validation hoạt động	6
Hình 6. Cây quyết định Decision Tree	7
Hình 7. Mô hình Random Forest	9
Hình 8. Khung xương và các điểm khớp được Visualize	10
Hình 9. Tập dữ liệu pose_data.csv	12
Hình 10. Hiển thị số lượng mẫu bộ dữ liệu mới	13
Hình 11. Ma trận nhầm lẫn	16
Hình 12. Demo mô hình hoạt động	17

CHƯƠNG 1. GIỚI THIỆU

Chương I sẽ nêu sơ lược về bài toán đặt ra trong đề án và hướng giải quyết bài toán.

1.1 Tổng quan bài toán

Phân loại tư thế cơ thể người (human pose classification) là một bài toán phổ biến và các phương pháp của bài toán này đang ngày càng được sử dụng rộng rãi để xử lý nhiều vấn đề hiện nay. Với mong muốn tìm hiểu về cách hoạt động cũng như các ứng dụng thực tiễn của các phương pháp này, em quyết định sử dụng một trong các phương pháp phổ biến hiện nay là sử dụng framework MediaPipe đi kèm với mô hình học máy Random Forest để thực hiện đề tài “Phân loại động tác tập Gym sử dụng MediaPipe và Random Forest”. Việc thực hiện đề tài này sẽ là tiền đề để em có thể tìm tòi và phát triển, ứng dụng các phương pháp khác nhau trong tương lai nhằm giải quyết các vấn đề trong nhiều lĩnh vực đa dạng.

1.2 Hướng giải quyết bài toán

Trong phạm vi đề án này, bộ dữ liệu đầu vào của bài toán là do chính em tự tạo ra thông qua thu thập dữ liệu các điểm khớp đặc trưng (landmarks) khi em thực hiện các tư thế tập gym. Em sử dụng MediaPipe để có thể dự đoán được khung xương, các điểm khớp và các dữ liệu về chúng trực tiếp qua Webcam laptop. Dựa trên bộ dữ liệu được tạo ra, em thực hiện tăng cường dữ liệu thông qua hai phương pháp Jitter (thêm nhiễu nhỏ) và Mirror (lật trái-phải). Sau khi thu được bộ dữ liệu đã tăng cường bao gồm dữ liệu gốc và dữ liệu tăng cường, em bắt đầu phân chia dữ liệu bằng Train/Test Split, huấn luyện mô hình thông qua mô hình học máy Random Forest và đánh giá mô hình bằng kỹ thuật K-fold Cross-Validation với các chỉ số Accuracy, Precision, Recall, Confuse Matrix, Cross_val_score và OOB score để kịp thời đưa ra chỉnh sửa nếu có vấn đề. Cuối cùng em sẽ sử dụng mô hình đó để thực hiện phân loại động tác hiện tại của người tập gym trực tiếp thông qua Webcam laptop hoặc gián tiếp thông qua video và đếm số rep tập của một người cho từng động tác.

1.3 Bố cục báo cáo

Bố cục của báo cáo:

- CHƯƠNG 1: GIỚI THIỆU
- CHƯƠNG 2: CƠ SỞ LÝ THUYẾT
- CHƯƠNG 3: HUẤN LUYỆN MÔ HÌNH VÀ ĐÁNH GIÁ KẾT QUẢ

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

Chương 2 sẽ đưa ra thông tin về thư viện MediaPipe, cơ sở lý thuyết về các phương pháp tăng cường dữ liệu, phân chia dữ liệu, mô hình học máy Random Forest, và các chỉ số đánh giá chất lượng mô hình đã được sử dụng trong dự án.

2.1 Tổng quan về MediaPipe

Trong bài toán nhận diện tư thế người, việc xác định chính xác vị trí các bộ phận trên cơ thể (như vai, khuỷu tay, đầu gối, cổ tay, cổ chân,...) là yếu tố quan trọng giúp mô hình phân loại hoạt động hiệu quả. Trong dự án này, nhóm em sử dụng MediaPipe Pose – một giải pháp ước lượng tư thế cơ thể (Pose Estimation) do Google phát triển, thuộc thư viện MediaPipe mã nguồn mở.

MediaPipe Pose là một pipeline học sâu thời gian thực, gồm hai giai đoạn chính:

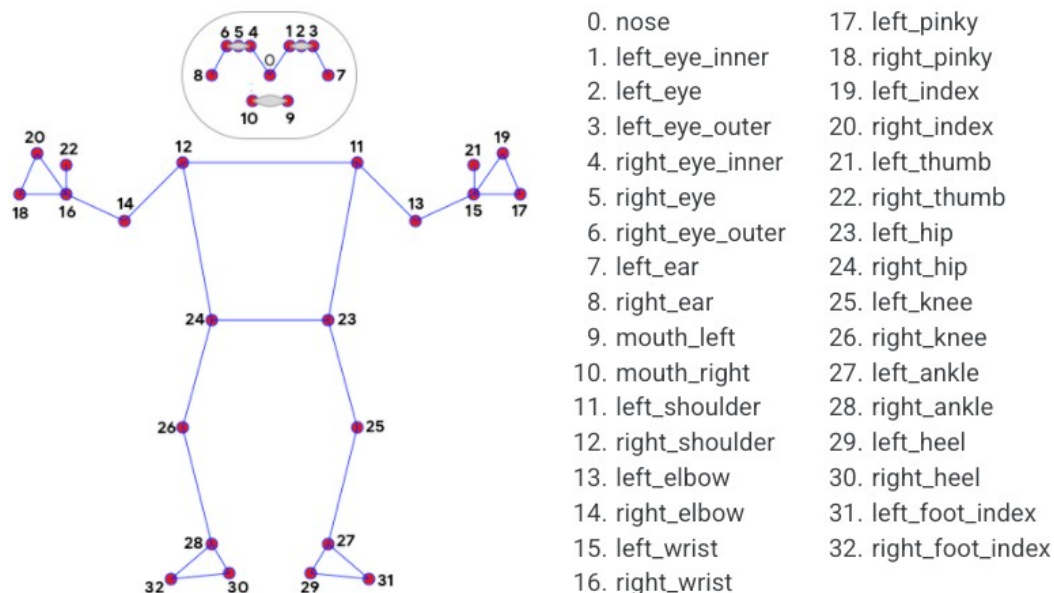
Giai đoạn 1: Phát hiện người trong khung hình video hoặc ảnh.

Giai đoạn 2: Dự đoán tọa độ của 33 điểm mốc trên cơ thể người (keypoints), tương ứng với các vị trí giải phẫu như đầu, cổ, vai, khuỷu tay, hông, đầu gối, cổ chân, bàn tay và bàn chân.

Mỗi keypoint mà MediaPipe phát hiện được cung cấp dưới dạng 4 thông tin:

- x, y : Tọa độ điểm trên khung hình (giá trị được chuẩn hóa từ 0 đến 1).
- z : Độ sâu tương đối của điểm so với camera.
- $visibility$: Độ tin cậy của điểm, phản ánh khả năng điểm đó bị che khuất hay không (giá trị từ 0 đến 1).

Một trong những ưu điểm nổi bật của MediaPipe là có thể hoạt động real-time trên CPU, không cần GPU và không cần các marker vật lý gắn lên người như các phương pháp truyền thống. MediaPipe có thể tích hợp dễ dàng với thư viện OpenCV để hiển thị kết quả lên khung hình trực tiếp.



Hình 1. Cấu trúc khung xương và các điểm khớp đặc trưng trong human pose của MediaPipe

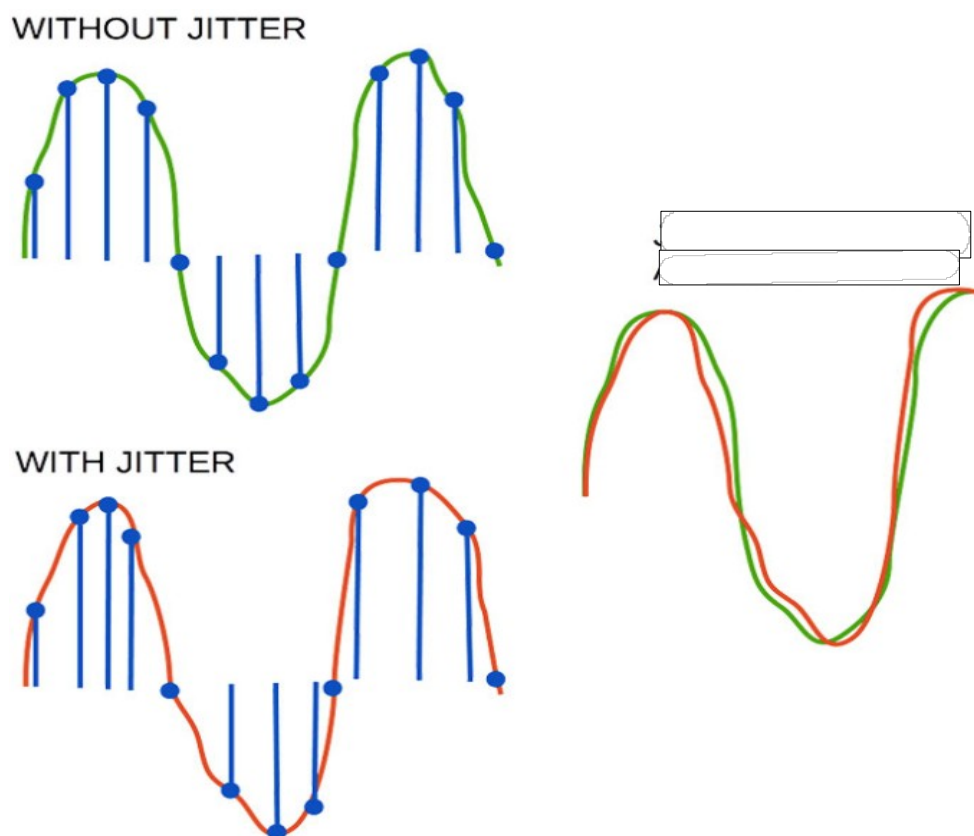
2.2 Các kỹ thuật tăng cường dữ liệu được sử dụng

Trong các bài toán học máy, đặc biệt là khi sử dụng mô hình phân loại, kích thước và độ đa dạng của tập dữ liệu huấn luyện có ảnh hưởng rất lớn đến khả năng tổng quát của mô hình. Tuy nhiên, trong thực tế, việc thu thập dữ liệu từ nhiều người, nhiều điều kiện ánh sáng, góc quay khác nhau là tốn thời gian và công sức. Để giải quyết vấn đề này, em đã áp dụng hai kỹ thuật tăng cường dữ liệu (Data Augmentation) đơn giản nhưng hiệu quả: Jitter và Mirror.

2.2.1. Kỹ thuật Jitter (thêm nhiễu nhỏ ngẫu nhiên)

Jitter là phương pháp thêm một lượng nhiễu nhỏ ngẫu nhiên vào dữ liệu gốc nhằm mô phỏng các biến đổi nhỏ. Ở dự án này, các dữ liệu tư thế gồm các điểm tọa độ (x, y, z, visibility) đại diện cho vị trí của các keypoints đặc trưng cho cơ thể người. Thông qua Jitter, chúng em có thể khiến các dữ liệu này dao động nhỏ trong một khoảng nhất định (chỉ từ $\pm 1-5\%$), từ đó mô phỏng được chuyển động biến đổi nhỏ trên cơ thể người khi thực hiện các động tác.

Với kỹ thuật trên, bộ dữ liệu của em được tăng cường thêm các mẫu dữ liệu đa dạng hơn. Vì là biến thể của mẫu dữ liệu cũ, các mẫu dữ liệu này không ảnh hưởng đến bản chất của tư thế nên sẽ không gây ra trường hợp bản chất mẫu quá sai khác với nhãn. Việc sử dụng kỹ thuật Jitter cũng đồng thời giúp cho việc huấn luyện mô hình sau này của em giảm bớt tình trạng overfitting – tình trạng học quá khớp.



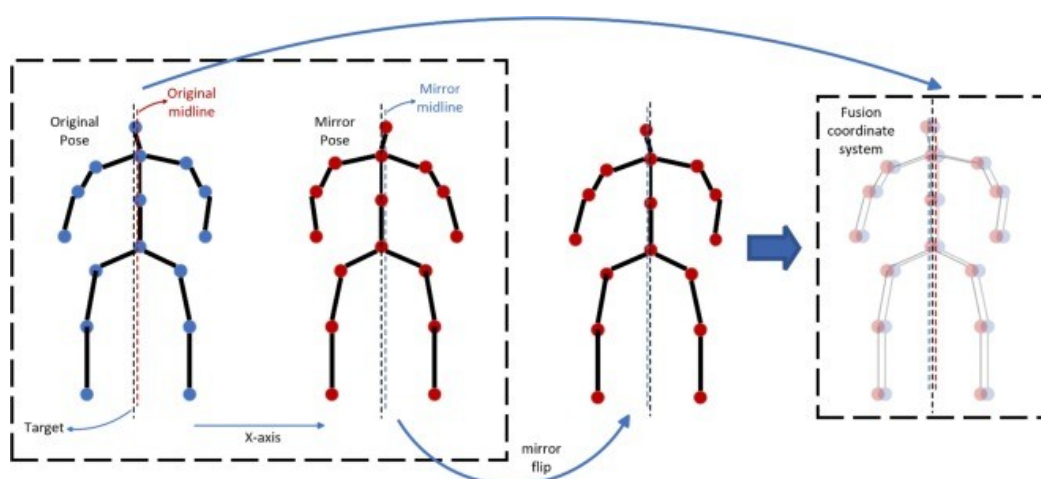
Hình 2. Minh họa cách hoạt động của Jitter

2.2.2. Kỹ thuật Mirror (lật trái-phải)

Mirror là kỹ thuật tạo ra bản sao của dữ liệu bằng cách lật đối xứng cơ thể người theo trục dọc (giống như soi gương). Cụ thể trong dự án này, các điểm mốc bên trái và bên phải (ví dụ: vai trái và vai phải) sẽ hoán đổi vị trí và giá trị cho nhau.

Để làm được điều này, chúng ta cần phải xác định những điểm keypoints thuộc bên trái và bên phải. Sau đó hoán đổi các giá trị (x , y , z , visibility) của các cặp điểm đối xứng nhau. Thư viện MediaPipe đã hỗ trợ xác định hướng các điểm keypoints thông qua tên các điểm (LEFT và RIGHT).

Kỹ thuật Mirror giúp mô hình học được tư thế người dù đứng trái hay phải, giúp tăng gấp đôi dữ liệu mà không làm thay đổi bản chất tư thế.



Hình 3. Minh họa cách hoạt động của kỹ thuật Mirror

Hai kỹ thuật tăng cường dữ liệu đơn giản là Jitter và Mirror đã giúp nâng cao khả năng học và tổng quát hóa của mô hình, đồng thời tiết kiệm thời gian thu thập dữ liệu mới, làm giàu bộ dữ liệu. Ngoài ra, cả hai phương pháp này đều giúp mô hình tránh tình trạng overfitting khi mà mô hình “học thuộc lòng” chứ không “học hiểu”, dẫn đến các phán đoán sai lầm khi dự đoán các đầu vào mới khác với dữ liệu được huấn luyện.

2.3 Các phương pháp phân chia và đánh giá dữ liệu được sử dụng

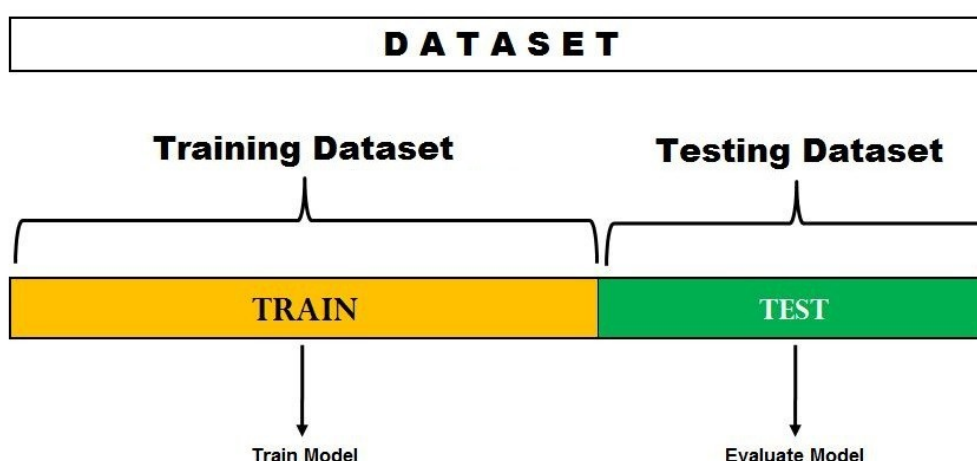
Trong các bài toán học máy, việc đánh giá mô hình trên dữ liệu chưa từng thấy trong quá trình huấn luyện là vô cùng quan trọng để kiểm tra khả năng tổng quát hóa. Để làm điều này, dữ liệu ban đầu thường được chia thành nhiều phần với các mục đích khác nhau. Hai kỹ thuật phổ biến nhất là: Train/Test Split và K-Fold Cross-Validation.

2.3.1. Kỹ thuật Train/Test Split

Đây là cách chia dữ liệu đơn giản và thường được sử dụng nhất. Đúng như cái tên của mình, kỹ thuật này chia bộ dữ liệu đầu vào thành 2 phần: tập huấn luyện (Training set) và tập kiểm tra (Test set). Ngoài ra còn có thể có chia ra tập xác thực (Validation set)

- **Training set:** tập này dùng để huấn luyện mô hình nắm được các đặc trưng của dữ liệu đầu vào.
- **Test set:** đây là tập dùng để kiểm tra chất lượng của mô hình được huấn luyện qua Training set.
- **Validation set:** tập dữ liệu này có thể có hoặc không, dùng để kiểm thử độ chính xác của mô hình trong quá trình huấn luyện, từ đó tinh chỉnh mô hình, hướng tới mô hình chính xác nhất.

Nếu chỉ chia 2 tập Train và Test, tỉ lệ chia tiêu chuẩn thường sẽ là 80/20. Nếu có sử dụng thêm tập Validation, tỉ lệ Train/Test/Validation thường là 70/15/15.



Hình 4. Dữ liệu gốc được chia thành 2 tập dữ liệu Train và Test

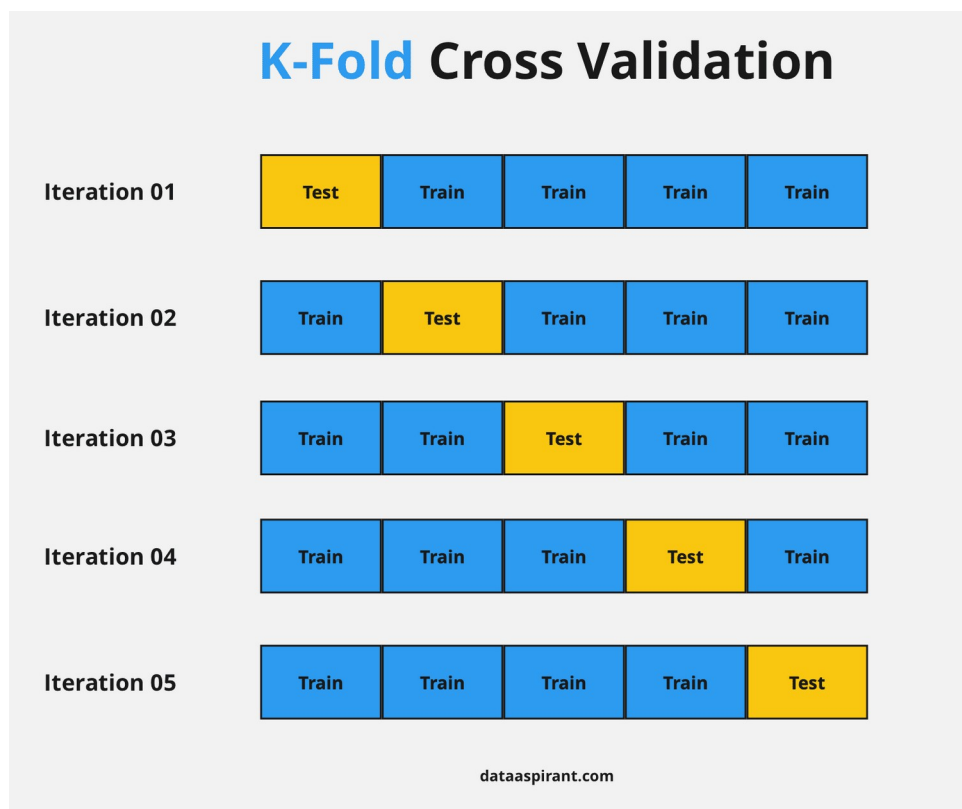
Ưu điểm của kỹ thuật Train/Test Split là nhanh, rất dễ thực hiện và phù hợp khi tập dữ liệu đầu vào có số mẫu lớn. Tuy nhiên, hạn chế của kỹ thuật này là dễ bị lệch dữ liệu nếu tập dữ liệu không đồng đều, có khả năng bị overfitting cao nếu tập dữ liệu không đa dạng, các mẫu quá giống nhau. Kết quả đánh giá dựa vào các metrics như

Accuracy, Precision, Recall cũng như Confusion Matrix và phụ thuộc vào cách chia cụ thể.

2.3.2. Kỹ thuật K-fold Cross-Validation

K-fold Cross-Validation là một phương pháp phân chia và đánh giá một cách tổng quát hơn so với Train/Test Split. Quá trình phân chia và đánh giá diễn ra như sau:

- Đầu tiên, bộ dữ liệu ban đầu sẽ được chia làm K phần có số mẫu như nhau.
- Thực hiện K lần vòng lặp, trong mỗi vòng lặp đó sẽ sử dụng K-1 phần để huấn luyện mô hình (Train set) và 1 phần còn lại để kiểm tra mô hình (Test set)
- Với mỗi vòng lặp đó, ta sẽ tính các thông số Accuracy, Precision, Recall,...
- Sau khi hoàn thành tất cả các vòng lặp, ta sẽ thu được mô hình được huấn luyện và kiểm tra K lần trên các phần khác nhau của bộ dữ liệu. Ngoài ra ta cũng thu được trung bình các thông số Accuracy, Precision, Recall,... qua tất cả các vòng.



Hình 5. Cách kỹ thuật K-fold Cross-Validation hoạt động

Ưu điểm nổi bật của kỹ thuật K-fold Cross-Validation là mô hình được huấn luyện và kiểm tra rất kỹ càng, từ đó giảm rủi ro lệch dữ liệu, đồng thời cho kết quả đánh giá khách quan và ổn định hơn, đặc biệt tránh tình trạng overfitting rất hiệu quả. Tuy nhiên, vì phải huấn luyện K lần, kỹ thuật này sẽ yêu cầu nhiều thời gian hơn so với Train/Test Split.

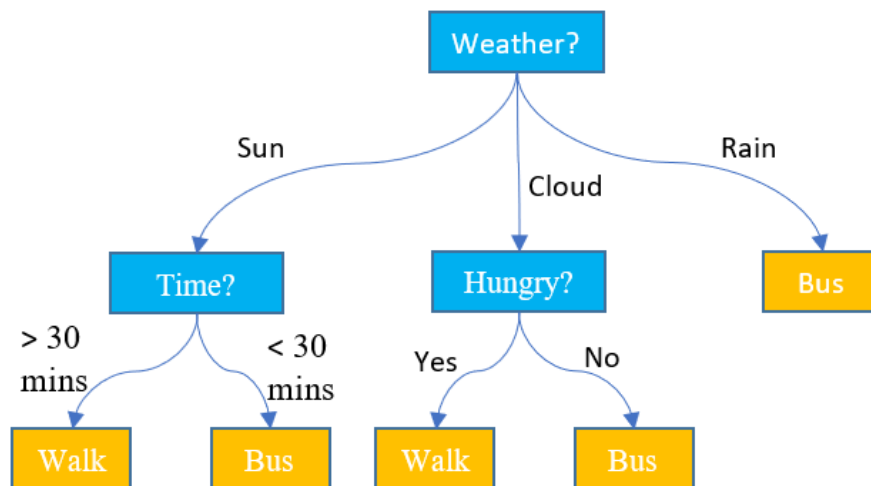
2.4 Cơ sở lý thuyết mô hình Random Forest

Random Forest là một thuật toán học máy thuộc nhóm mô hình học có giám sát (Supervised learning), được sử dụng rộng rãi cho cả bài toán phân loại

(Classification) và hồi quy (Regression). Là tập hợp của nhiều cây quyết định (Decision Tree), Random Forest đem lại khả năng phán đoán chính xác và tổng quát hóa tốt hơn so việc sử dụng một mô hình đơn lẻ.

2.4.1. Giải thuật Decision Tree

Cây quyết định (Decision Tree) là là một mô hình Supervised learning, có thể được áp dụng vào cả hai bài toán classification và regression. Mỗi một nút trong (internal node) tương ứng với một biến; đường nối giữa nó với nút con của nó thể hiện một giá trị cụ thể cho biến đó. Mỗi nút lá đại diện cho giá trị dự đoán của biến mục tiêu, cho trước các giá trị của các biến được biểu diễn bởi đường đi từ nút gốc tới nút lá đó.



Hình 6. Cây quyết định Decision Tree

Cụ thể hơn:

- Mỗi **nút (node)** đại diện cho một điều kiện phân chia dựa trên đặc trưng của dữ liệu
- Mỗi **nhánh (branch)** tương ứng với kết quả của điều kiện
- **Lá (leaf node)** chứa kết quả phân loại hoặc giá trị dự đoán

Mô hình sẽ bắt đầu từ gốc, kiểm tra điều kiện ở từng nút, và đi xuống theo các nhánh cho đến khi gặp lá — nơi đưa ra quyết định cuối cùng.

Khi sử dụng mô hình Decision Tree, mô hình sẽ tìm đặc trưng và ngưỡng chia dữ liệu tốt nhất để phân tập dữ liệu mẫu thành các nhánh. Tùy vào loại thuật toán xây dựng cây quyết định, sẽ có các thông số đánh giá tương ứng. Với $\mathbf{p} = (p_1, p_2, \dots, p_n)$ là phân phối xác suất của một biến rời rạc x có thể nhận n giá trị khác nhau $x_1, x_2, \dots, x_n$, ta có:

- **Thuật toán ID3, C4.5:** các thuật toán này sẽ sử dụng hàm số Entropy đại diện cho chất lượng của ngưỡng chia dữ liệu. Ta có Entropy của phân phối này theo công thức:

$$H(p) = - \sum_{i=1}^n p_i \log(p_i) \quad [2.1]$$

Với quy ước $0\log(0) = 0$, giá trị này nằm trong khoảng 0 đến 1. Nếu tất cả các mẫu trong tập dữ liệu S thuộc về một lớp, thì Entropy sẽ bằng 0. Nếu một nửa số mẫu được phân loại là một lớp và một nửa còn lại thuộc một lớp khác, thì Entropy sẽ đạt mức cao nhất là 1. Để chọn tính năng tốt nhất để phân tách và tìm cây quyết định tối ưu, thuộc tính có lượng entropy nhỏ nhất nên được sử dụng.

- **Thuật toán CART:** thuật toán này sử dụng Gini Index để đánh giá việc phân chia node ở điều kiện có tốt không. Gini Index được tính như sau:

$$Gini = 1 - \sum_{i=1}^n (p_i)^2 \quad [2.2]$$

Giá trị Gini giao động từ 0 đến 1. Nếu $Gini = 0$, tức là một node chỉ chứa các mẫu của một nhãn duy nhất, cho thấy node rất tốt. Nếu Gini tiến đến gần giá trị 0,5 hoặc hơn, node có nhiều nhãn và sẽ kém hơn. Nếu $Gini = 1$, tức là node rất hỗn loạn, số lượng mẫu các nhãn phân bố đồng đều trong node. Mục tiêu của cây quyết định là chọn phép chia nào làm giảm Gini Index nhiều nhất.

Các thuật toán sẽ tiếp tục chia nhỏ dữ liệu cho đến khi đạt được một trong các điều kiện sau:

- Đạt đến độ sâu tối đa ($\max_depth$) là giá trị mà người sử dụng cài đặt nhằm tránh mô hình học quá kỹ, dẫn đến overfitting.
- Không còn đặc trưng nào khả thi để có thể phân chia tiếp.
- Số lượng mẫu tại nút quá nhỏ, không thể chia một cách hiệu quả được nữa.

Khi đó, các node dưới cùng mỗi nhánh trong cây sẽ là các leaf node, đại diện cho các kết quả phân loại hoặc giá trị dự đoán đầu ra của Decision Tree.

2.4.2. Random Forest

Một cây quyết định đơn lẻ hoạt động sẽ có nguy cơ rất cao bị overfitting. Để tránh điều này, người ta thường giới hạn độ sâu của cây. Tuy nhiên, nếu độ sâu của cây quyết định quá nông có thể sẽ dẫn đến bỏ sót những biến quan trọng, khiến mô hình dự đoán/phân loại không hiệu quả. Để giải quyết vấn đề này, ý tưởng tập hợp nhiều cây quyết định lại để tạo thành mô hình Random Forest đã được đưa ra.



Hình 7. Mô hình Random Forest

Về cơ bản, Random Forest là sự kết hợp giữa **Mô hình kết hợp (ensemble model)** và **Lấy mẫu tái lập (bootstrapping)**.

Mô hình kết hợp trong Random Forest chính là việc kết hợp nhiều Decision Tree trong một mô hình. Mỗi Decision Tree sẽ hoạt động độc lập, đưa ra dự đoán/phân loại của mình. Với bài toán phân loại, kết quả phân loại của Random Forest sẽ dựa vào nhận được nhiều Decision Tree chọn nhất. Còn với bài toán hồi quy, giá trị dự đoán được đưa ra sẽ là trung bình của tất cả các kết quả của các Decision Tree.

Lấy mẫu tái lập là phương pháp tạo ra các tập dữ liệu con để xây dựng các Decision Tree trong mô hình Random Forest. Giả sử ta có một tập dữ liệu **A** có N mẫu. Thực hiện nhặt M lần các mẫu từ tập **A** và bỏ vào một tập dữ liệu con, ta sẽ có được một tập dữ liệu con có M mẫu. Tập dữ liệu con này cho phép các phần tử bên trong nó có thể được lặp lại. Với B tập dữ liệu con như thế được tạo ra, ta gọi quá trình này là **bỏ túi (Bagging)**. Sẽ có B Decision Tree được tạo ra dựa trên B tập dữ liệu con tương ứng. Bởi vì các phần tử trong một tập con có thể được lặp lại, sẽ có một số mẫu tồn tại trong tập dữ liệu **A** ban đầu nhưng không có trong B tập dữ liệu con. Đây là những mẫu chưa được bỏ túi và chúng ta gọi đó là **nằm ngoài túi (Out of Bag)**. Các mẫu OOB này có thể được tận dụng để ước lượng độ chính xác, vai trò tương tự như một Test set cho chính các Decision Tree, giúp đánh giá hiệu quả của Random Forest mà không cần chia nhỏ dữ liệu gốc. Nếu training accuracy cao, nhưng OOB score thấp, có thể mô hình đang overfit dữ liệu huấn luyện. Do đó, OOB giúp theo dõi chất lượng mô hình ngay trong quá trình huấn luyện.

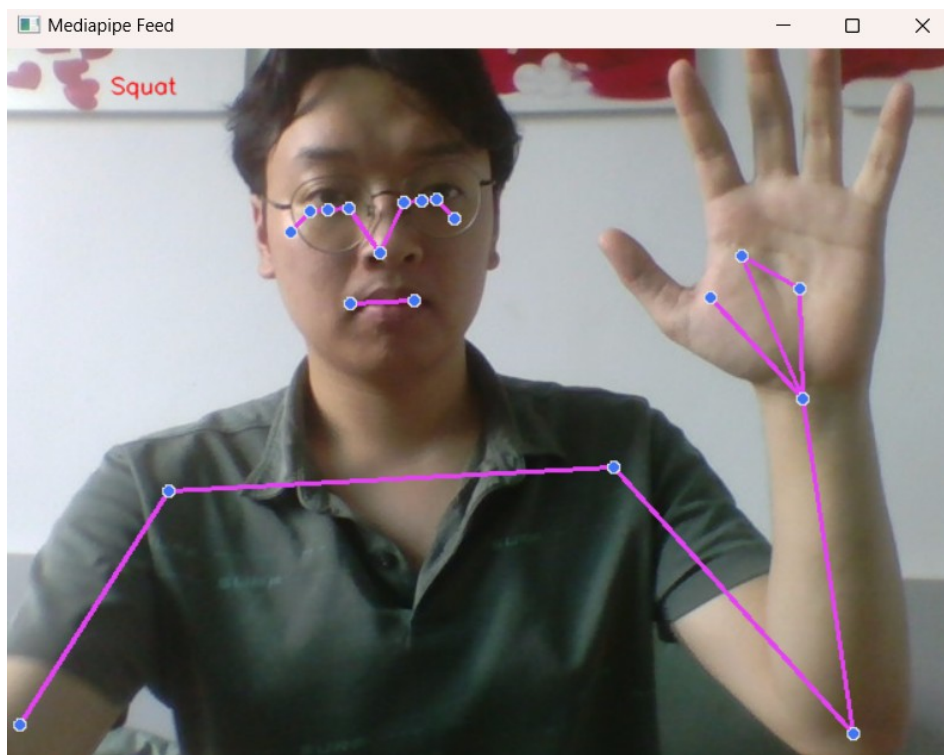
CHƯƠNG 3: HUẤN LUYỆN MÔ HÌNH VÀ ĐÁNH GIÁ KẾT QUẢ

Chương 3 sẽ nêu quá trình thu thập dữ liệu, tăng cường dữ liệu, huấn luyện mô hình, đánh giá kết quả sau khi huấn luyện và sử dụng mô hình, đồng thời chương này cũng nêu các khó khăn khi làm đề tài mà em gặp phải, và hướng phát triển của đề tài trong tương lai.

3.1 Thu thập dữ liệu

Để huấn luyện mô hình phân loại tư thế tập Gym, em sẽ cần thu thập các dữ liệu về tọa độ các điểm khớp keypoints trên cơ thể người trong tư thế đó, đồng thời gán nhãn cho các mẫu dữ liệu đó. Để làm được điều này, em quyết định tự tạo một bộ dữ liệu của riêng mình. Các bước tự tạo dữ liệu như sau:

[1] Đầu tiên, em sử dụng mô hình Pose Estimation của MediaPipe để có thể lấy được tọa độ các điểm khớp keypoints (33 điểm) trên cơ thể người trong thời gian thực thông qua Webcam laptop. Hình ảnh khung xương, keypoints được visualize lên cửa sổ Webcam trên màn hình laptop. Em cũng xác định bộ dữ liệu sẽ có 3 class bao gồm: “**Squat**”, “**PushUp**” và “**Bicep Curl**”.



Hình 8. Khung xương và các điểm khớp được Visualize

[2] File pose_data.csv được khởi tạo và có thể được ghi thêm nội dung. Bởi vì MediaPipe cung cấp 33 điểm keypoints mỗi mẫu dữ liệu và 4 dữ liệu tọa độ x,

y, z, visibility cho mỗi keypoints nên sẽ có tổng cộng $33 \times 4 = 132$ cột dữ liệu. Cùng với 1 cột label, pose_data.csv sẽ có tổng cộng 133 cột dữ liệu.

[3] Dữ liệu sẽ được thu thập và gán nhãn đồng thời. Với label mặc định là “Squat”, nhãn hiện tại có thể được thay đổi bằng các phím “1”, “2”, “3” đã được cài đặt tương ứng với các nhãn “Squat”, “PushUp” và “Bicep Curl”. Khi khung xương và các keypoints đang được visualize trên cửa sổ Webcam, mẫu dữ liệu sẽ được gán nhãn là nhãn tương ứng hiện tại và ghi lại vào file pose_data.csv qua từng frame được lựa chọn. Việc lựa chọn frame được ghi lại sẽ cho chính em quyết định thủ công bằng cách click chuột trái nhằm tránh thu thập nhầm dữ liệu không tương thích với nhãn. Đặc biệt dữ liệu sẽ chỉ được ghi lại khi có đủ 33 điểm keypoints được mô hình Pose Estimation của MediaPipe tìm thấy. Điều này nhằm tránh thu thập thiếu dữ liệu của các điểm keypoint không có trong khung hình, dẫn đến mẫu dữ liệu bị sai hoặc thiếu.

[4] Sau khi hoàn tất các bước chuẩn bị trên, em bắt đầu thu thập dữ liệu. Với mỗi tư thế, em thu thập 50 mẫu dữ liệu. Trong 50 mẫu này sẽ bao gồm dữ liệu frame của 3 giai đoạn trong một tư thế. Ví dụ như động tác Push up, em thu thập dữ liệu của frame khi cơ thể hạ thấp hoàn toàn, frame khi cơ thể đang hơi hạ xuống và frame của cơ thể khi đang chống thẳng tay lên. Điều này là vì các động tác được tạo thành bởi các chuyển động liên tục của người tập, trong khi dữ liệu thu được chỉ là từ các frame tĩnh. Việc thu thập frame của 3 giai đoạn trong một động tác tập phần nào có thể mô phỏng được chuỗi chuyển động phiên bản tối giản của động tác đó, giúp tăng hiệu quả nhận diện và phân loại sau này. Bên cạnh đó, em cũng thu dữ liệu khi thực hiện tư thế ở nhiều hướng khác nhau cũng như các mức ánh sáng khác nhau, tránh giữ nguyên vị trí nhằm tăng sự đa dạng của tập dữ liệu, giảm overfit.

[5] Sau khi thu thập dữ liệu hoàn tất, em thu được một tập dữ liệu gồm tổng 150 mẫu với 50 mẫu cho mỗi class. Tập dữ liệu này được lưu trong file pose_data.csv, có dạng như **Hình 9**


```

1 label, x0, y0, z0, v0, x1, y1, z1, v1, x2, y2, z2, v2, x3, y3, z3, v3, x4, y4, z4, v4, x5, y5, z5, v5, x6, y6, z6, v6, x7, y7, z7, v7, x8, y8, z8, v8, x9, y9, z9, v9, x10, y10, z10, v10, x11, y11, z11, v11, x12, ✓
2 Squat, 0.4760478138923645, 0.42311590909957886, -0.4129278063774109, 0.9994242191314697, 0.48663678765296936, 0.39691635966300964, -0.3990020453929901, 0.9991005063056946, 0.
3 Squat, 0.3797328472137451, 0.4245092272758484, -0.45562297105789185, 0.9995141625404358, 0.38711196184158325, 0.3949885070323944, -0.46704626083374023, 0.999337936019897, 0.
4 Squat, 0.3220406174659729, 0.4204564392566681, -0.18393191695213318, 0.9997106194496155, 0.3257766366004944, 0.39038485288619995, -0.19084593653678894, 0.9998040199279785, 0.
5 Squat, 0.30676597356796265, 0.44604364037513733, 0.02713223733007908, 0.9997864961624146, 0.3027017414569855, 0.4192832112312317, -0.0013822481269016862, 0.99988853931427, 0.
6 Squat, 0.3244383633136749, 0.4764508605003357, 0.1665070354938507, 0.9969488978359525, 0.31756144762039185, 0.4539140462875366, 0.13203869760036469, 0.9985442161560059, 0.318
7 Squat, 0.3937091827392578, 0.42745551466941833, 0.378288209438324, 0.9692841172218323, 0.38578569088836365, 0.4202412962913513, 0.3563280403614044, 0.9873100519180298, 0.3815
8 Squat, 0.49841904640197754, 0.43265634775161743, 0.5206061601638794, 0.9617863297462463, 0.49220237135887146, 0.4173169434070587, 0.4965856671333313, 0.9759002923965454, 0.48
9 Squat, 0.6159371733665466, 0.4936583638191223, 0.18294325470924377, 0.9984174370765686, 0.6198359131813049, 0.4736655056476593, 0.18811708688735962, 0.9986637830734253, 0.618
10 Squat, 0.4847939610481262, 0.4280274212360382, -0.45941162109375, 0.9999874234199524, 0.4958071708679199, 0.39767417311668396, -0.44367408752441406, 0.9999782443064657, 0.5022
11 Squat, 0.33198612928390503, 0.42447832226753235, -0.32089701294898987, 0.9995879530906677, 0.33707207441329956, 0.39594078063964844, -0.33709877729415894, 0.9996064901351929
12 Squat, 0.31514865159988403, 0.3834529519081116, 0.021060863509774208, 0.9996493458747864, 0.3176676630973816, 0.3592846989631653, -0.010450865142047405, 0.9997382760047913, 0.
13 Squat, 0.32547810673713684, 0.41738805174827576, 0.12893907725811005, 0.9988030791282654, 0.3270014524459839, 0.3952484130859375, 0.09493973851203918, 0.9993315935134888, 0.3
14 Squat, 0.4634962272644043, 0.3948652446269989, 0.4207053780555725, 0.9717467427253723, 0.4361129701137543, 0.38180068135261536, 0.3893391788005829, 0.9861776232719421, 0.4321
15 Squat, 0.5297232866287231, 0.41456741094589233, 0.5858942270278931, 0.99629771806526184, 0.5242170691490173, 0.40006887912750244, 0.56566232424285583, 0.9980326890945435, 0.518
16 Squat, 0.6329482793807983, 0.43519535660743713, 0.21187293529510498, 0.9994776248931885, 0.630497395992279, 0.41086798906326294, 0.22046223282814026, 0.9996682405471802, 0.62
17 Squat, 0.6938189268112183, 0.41360175609588623, -0.06035936623811722, 0.9998916983604431, 0.6932787895202637, 0.3887994587421417, -0.03916376084089279, 0.9999313354492188, 0.
18 Squat, 0.6311386823654175, 0.35859501361846924, -0.567997395992279, 0.9999368190765381, 0.6381521224975586, 0.33477723598400225, -0.5423394441604614, 0.9999468922615051, 0.64
19 Squat, 0.5778822302818298, 0.37125903367996216, -0.5797365307807922, 0.9998211860656738, 0.5911337733268738, 0.3392539620399475, -0.5547091364860535, 0.9998037219047546, 0.59
20 Squat, 0.46375638246536255, 0.3509191572666168, -0.5605205297470093, 0.9999667406082153, 0.4756692051887512, 0.3155900537967682, -0.5426943302154541, 0.9999494552612305, 0.48
21 Squat, 0.3053235113620758, 0.36641761660575867, -0.425741583108902, 0.9996468424797058, 0.3106149435043335, 0.3271419107913971, -0.45074182748794556, 0.9995688199996948, 0.33
22 Squat, 0.4916626811027527, 0.25658419728279114, -0.5225257873535156, 0.9999983906745911, 0.5027283430099487, 0.23359237611293793, -0.5053752064704895, 0.9999962449073792, 0.3
23 Squat, 0.3641454577445984, 0.2466132938861847, -0.42659512162208557, 0.99999582764748657, 0.3713334798812866, 0.21795107424259186, -0.42155131697654724, 0.9999915957450867, 0.
24 Squat, 0.3135816156864166, 0.25897473096847534, -0.009963789954781532, 0.9997410774230957, 0.31414031982421875, 0.23475784063339233, -0.038611043244600296, 0.999745190143588
25 Squat, 0.3644034266471863, 0.298312783241272, 0.28927281498908997, 0.9990283250808716, 0.3637602925300598, 0.2724082171919617, 0.2556818425655365, 0.9991275668144226, 0.3643
26 Squat, 0.4677422046661377, 0.2901519238948822, 0.7340325713157654, 0.9607904553413391, 0.4593806862831116, 0.2794671356678009, 0.7070730328559875, 0.9626390337944031, 0.45418

```

Hình 9. Tập dữ liệu pose_data.csv

Việc thu thập dữ liệu này cần phải vô cùng cẩn thận bởi đây là bộ dữ liệu gốc, nếu chất lượng không tốt sẽ ảnh hưởng trực tiếp đến bộ dữ liệu tăng cường dùng để huấn luyện cũng như kiểm tra sau này, dẫn đến việc chất lượng của mô hình không được đảm bảo. Sau khi hoàn thành việc thu thập dữ liệu, em bắt tay vào tăng cường dữ liệu, làm giàu tính đa dạng cho bộ dữ liệu.

3.2 Tăng cường dữ liệu

Với tập dữ liệu 150 mẫu chia đều cho 3 class, em nhận thấy rằng số lượng thể này là quá ít, khiến cho việc chia Train/Test Split hoặc K-fold thiếu hiệu quả do các tập con quá nhỏ, dễ dẫn đến mô hình sau huấn luyện bị overfitting. Vì vậy, em quyết định thực hiện tăng cường dữ liệu dựa trên 150 mẫu dữ liệu gốc của nhóm. Các kỹ thuật tăng cường em quyết định sử dụng cho toàn bộ 150 mẫu trên tập dữ liệu gốc là kỹ thuật **Jitter (thêm nhiễu nhỏ)** và **Mirror (lật trái-phải)**.

Về kỹ thuật Jitter, em đã thêm một lượng nhiễu nhỏ ngẫu nhiên trong khoảng -3% đến 3% vào tất cả 132 dữ liệu tọa độ của 33 điểm keypoints trong một mẫu dữ liệu. Sau nhiều lần thử nghiệm, việc chọn mức dao động trong khoảng 3% cho thấy sự hiệu quả khi mô phỏng chuyển động nhẹ của cơ thể người tập luyện, đồng thời cũng ít có nguy cơ xuất hiện lỗi tư thế như với mức dao động lớn hơn. Sau khi hoàn thành, em thu được thêm 150 mẫu dữ liệu mới chia đều cho 3 class.

Tiếp theo là kỹ thuật Mirror. Kỹ thuật này đơn giản chỉ đổi chỗ các điểm keypoints đối xứng trái phải để tạo thành mẫu dữ liệu mới. Với việc MediaPipe hỗ trợ xác định các keypoints trái phải tương ứng với nhau thông qua tên, việc xác định, ghép cặp và hoán đổi lần lượt các giá trị tọa độ giữa 2 keypoints đối xứng là vô cùng dễ dàng. Sau khi hoàn thành, em thu được thêm 150 mẫu dữ liệu mới.

Sau khi hoàn thành 2 kỹ thuật trên, em sẽ thực hiện gộp 3 bộ dữ liệu bao gồm 2 bộ dữ liệu mới và bộ dữ liệu gốc vào thành một bộ dữ liệu hoàn chỉnh gồm 450 mẫu dữ liệu và được đặt trong file pose_data_augmented.csv. Bộ dữ liệu này sẽ là bộ dữ liệu dùng để huấn luyện cũng như kiểm tra và xác thực cho mô hình.

```

label      450
x0         450
y0         450
z0         450
v0         450
...
v31        450
x32        450
y32        450
z32        450
v32        450
Length: 133, dtype: int64

```

Hình 10. Hiển thị số lượng mẫu bộ dữ liệu mới

3.3 Phân chia dữ liệu và huấn luyện mô hình

Trước khi bắt đầu quá trình huấn luyện dữ liệu, bước đầu tiên em cần phải làm chính là tiền xử lý tập dữ liệu với việc tạo ra ma trận đặc trưng đầu vào (feature matrix) x và vector nhãn mục tiêu (target label) y bằng cách tách cột label ra khỏi phần còn lại của bộ dữ liệu. Sau đó, bởi vì mô hình không thể nhận biết các label dưới dạng chuỗi kí tự, em đã mã hóa các label thành dạng số 0,1 và 2 tương ứng với 3 nhãn “Squat”, “PushUp” và “Bicep Curl”, sử dụng bộ mô hình LabelEncoder của Scikit-learn. Từ đó, một mảng số nguyên $y_encoded = [0,1,2,...]$ được tạo ra, các giá trị phân tử đại diện cho các nhãn của bộ dữ liệu đầu vào.

```

df = pd.read_csv('pose_data_augmented.csv')
print(df.head())
print(df.count())

x = df.drop(labels="label", axis=1)
y = df["label"]

from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
y_encoded = le.fit_transform(y)

```

Sau bước tiền xử lý nhẹ, em bắt đầu phân chia bộ dữ liệu bằng kỹ thuật Train/Test Split và đánh giá bộ dữ liệu bằng kỹ thuật K-fold Cross-Validation để theo dõi và đảm bảo chất lượng mô hình. Việc kết hợp này đảm bảo đánh giá chất lượng của mô hình một cách chính xác khi mà mô hình được đánh bằng K-fold sẽ được đánh giá trên một tập Test chưa được gặp bao giờ đã được Train/Test tách riêng từ đầu. Mô hình học máy có giám sát RandomForestClassifier được gọi để làm mô hình huấn luyện:

```
from sklearn.ensemble import RandomForestClassifier
clf = RandomForestClassifier(oob_score=True, n_estimators=100, max_depth=5, random_state=42)
```

Với:

- **oob_score = True:** Dùng để xác định điểm đánh giá nội bộ OOB
- **n_estimators = 100:** Xác định số cây Decision Tree trong mô hình là 100 cây
- **max_depth = 5:** Kiểm soát độ sâu của các cây quyết định tối đa là 5, đề phòng tình trạng học quá kỹ dẫn đến overfitting.
- **random_state = 42:** là một tham số ngẫu nhiên, đóng vai trò đảm bảo có thể tái hiện lại kết quả huấn luyện. Nếu không có tham số này, trong mỗi lần chạy, dữ liệu huấn luyện sẽ thay đổi, dẫn đến độ chính xác mỗi lần cũng sẽ khác nhau.

a. Train/Test Split

Với kỹ thuật Train/Test Split, em quyết định phân chia bộ dữ liệu thành Train/Test với tỷ lệ 80/20. Đây là tỷ lệ thường được sử dụng với các bộ dữ liệu vừa và nhỏ, phù hợp với bộ dữ liệu của em, đồng thời có thể đánh giá mô hình một cách đáng tin cậy nhờ vào tập Train và Test đều có số mẫu đủ lớn lần lượt là 360 mẫu và 90 mẫu.

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(*arrays: x, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded)
```

Các tham số được truyền vào bao gồm:

- **test_size = 0,2:** tỷ lệ số mẫu cho tập Test là 20% tổng số mẫu của tập dữ liệu đầu vào.
- **random_state = 42:** chức năng tương tự như với RandomForestClassifier
- **stratify = y_encoded:** tham số này đảm bảo mỗi class sẽ được phân chia Train/Test đồng đều theo tỷ lệ đã chọn, tránh gom tất cả class vào rồi phân chia ngẫu nhiên.

b. K-fold Cross-Validation

Sau khi có được tập dữ liệu Train từ Train/Test Split, kỹ thuật K-fold Cross-Validation được sử dụng để đánh giá chất lượng mô hình trước khi huấn luyện chính thức trên tập Train.

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(clf, x_train, y_train, cv=5)
print("Cross-validation accuracy train set:", scores.mean())
```

Với:

- **clf:** là mô hình RandomForestClassifier được gọi ở trên
- **x_train, y_train:** là tập Train để K-fold huấn luyện nội bộ và đưa ra đánh giá

- **cv = 5**: nghĩa là K=5, tức là số lượng tập con được chia ra từ tập Train trong huấn luyện nội bộ là 5.

Cuối cùng, mô hình được huấn luyện chính thức trên tập Train và được đánh giá qua các thông số khác nhau. Ngoài ra, em tiến hành lưu mô hình và mã hóa nhãn bằng thư viện Joblib nhằm tái sử dụng mô hình đã huấn luyện trong file code chạy thử nghiệm mô hình.

```
clf.fit(x_train, y_train)

joblib.dump(clf, filename: "pose_classifier.pkl")
joblib.dump(le, filename: "label_encoder.pkl")
```

3.4 Đánh giá kết quả

Sau khi huấn luyện mô hình, các thông số đánh giá chất lượng mô hình cũng được tính toán nhằm đảm bảo mô hình có độ chính xác cao. Sau đây là kết quả thu được khi thực hiện tính toán các thông số đánh giá:

```
Cross-validation accuracy train set: 98.33333333333334
00B score: 98.33333333333333
số x_train = 360
số x_test = 90
Độ chính xác Accuracy là 98.89%
Precision score là 98.92%
Recall = 98.89%
Confusion matrix:
[[30  0  0]
 [ 0 30  0]
 [ 1  0 29]]
```

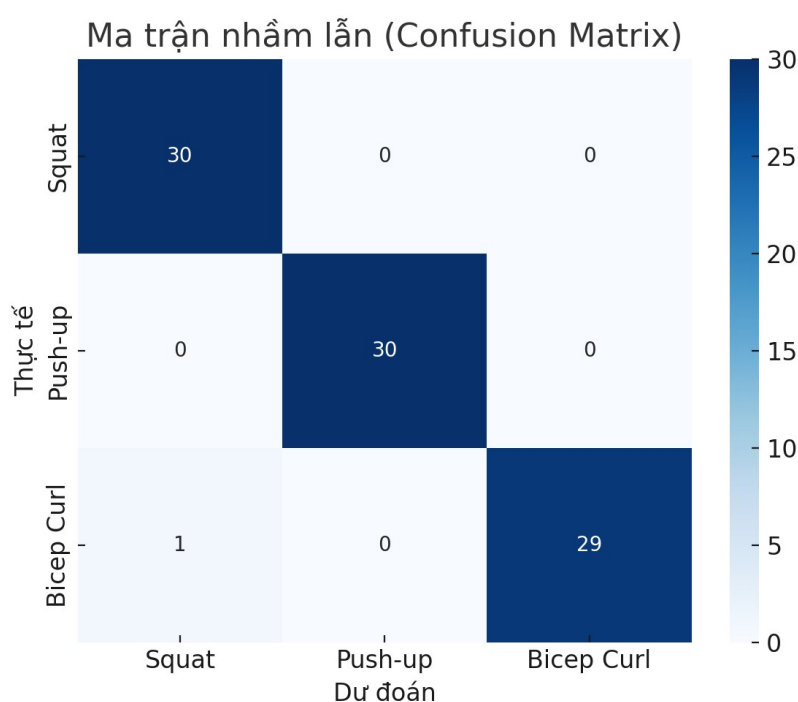
Cụ thể, ta có bảng:

Bảng 1. Các thông số đánh giá chất lượng mô hình

Accuracy (Test set)	Precision (Test set)	Recall (Test set)	Cross-val accuracy	Observed
------------------------	-------------------------	----------------------	-----------------------	----------

				r e
98.89%	98.92%	98.89%	98.34 %	9 8 . 3 3 %

Và Ma trận nhầm lẫn (Confusion Matrix):



Hình 11. Ma trận nhầm lẫn

Nhìn vào thông số **Bảng 1**, có thể thấy các thông số chất lượng khi thực hiện kiểm tra mô hình qua tập Test là rất ấn tượng. Với các chỉ số Accuracy, Precision và Recall có giá trị gần tương đương nhau, đều xấp xỉ 98,9%, có thể nhận xét rằng mô hình đang cân bằng và có độ chính xác rất cao.

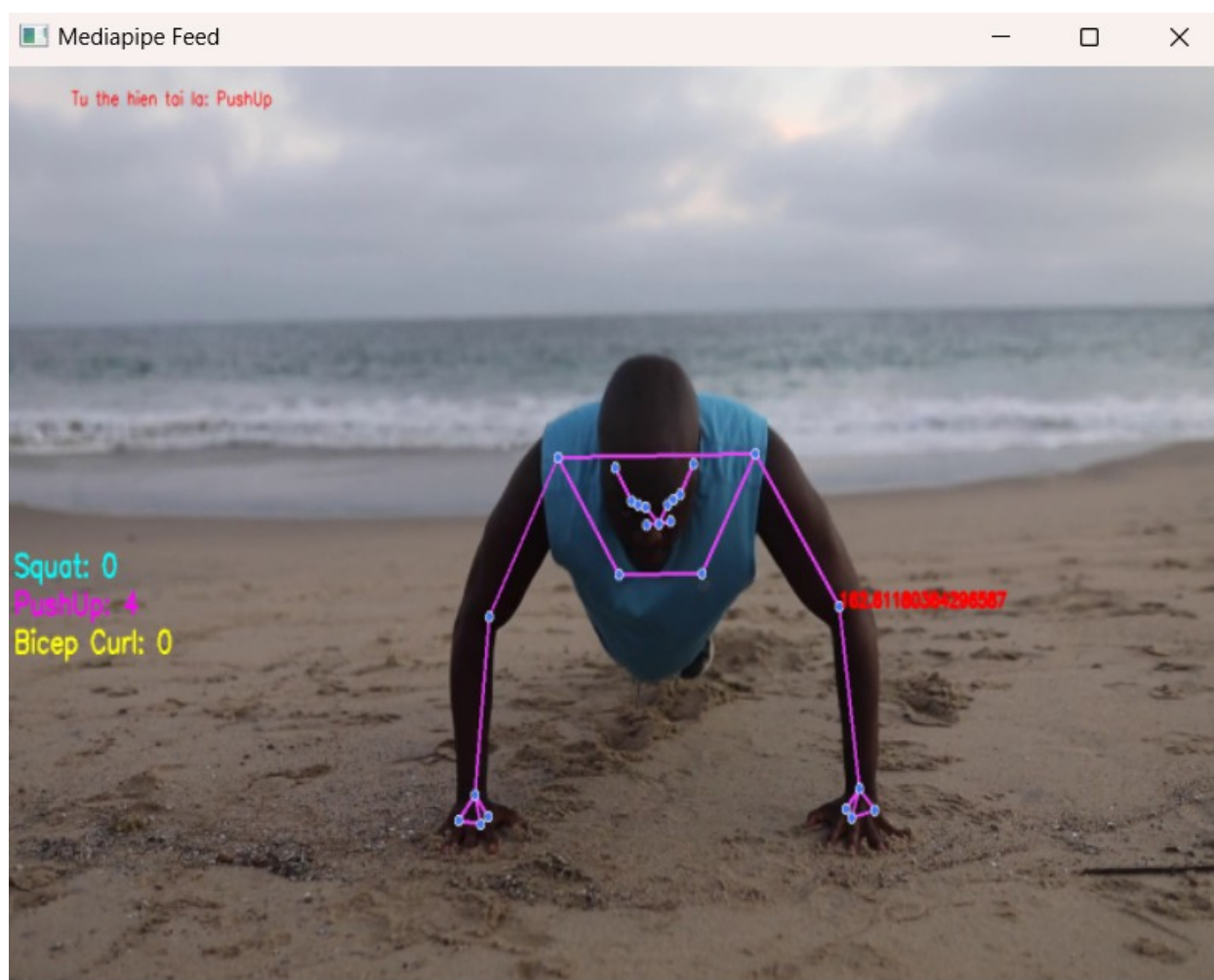
Bên cạnh đó, với việc cả 2 thông số Cross_validation_accuracy (sử dụng kỹ thuật đánh giá K-fold Cross-Validation với K=5) và OOB score (đánh giá nội bộ của mô hình RandomForestClassifier) đều có giá trị bằng 98,33% và gần bằng giá trị Accuracy trên tập Test, có thể đánh giá rằng mô hình đạt hiệu suất ổn định qua nhiều lần chia dữ liệu khác nhau, có tính nhất quán cao và đáng tin cậy. Điều này cũng phản ánh rằng Random Forest đã học tốt và không có dấu hiệu overfitting.

Về ma trận nhầm lẫn, các lớp đều dự đoán đúng tuyệt đối 2 class “Squat” và “PushUp” và chỉ sai 1 mẫu class “Bicep Curl” trên tổng số 90 mẫu trong tập Test,

cho thấy mô hình học không bị thiên vị. Một mẫu dự đoán sai (bị nhầm thành “Squat”) có thể là bắt nguồn từ sự tương đồng nhẹ giữa động tác Squat và Bicep Curl (cùng có thời điểm đứng thẳng, duỗi thẳng 2 tay).

Cần nhấn mạnh rằng các thông số trên là khách quan và có độ tin cậy cao dựa trên Confusion Matrix và đặc biệt là dựa trên việc tập dữ liệu Test được chia độc lập với tập Train ngay từ đầu thông qua kỹ thuật Train/Test Split, do đó mô hình chưa từng gặp qua các mẫu dữ liệu này trong quá trình huấn luyện với tập dữ liệu Train.

Để xem xét hiệu quả hoạt động của mô hình, em đã tạo một chương trình demo sử dụng mô hình để phân loại động tác tập gym, sử dụng MediaPipe cả với video có sẵn và phân loại theo thời gian thực thông qua Webcam laptop, đồng thời đếm số rep tập với mỗi động tác. Kết quả cho thấy mô hình hoạt động rất ổn định và phân loại đúng hầu hết các động tác tương ứng với các class đã được học.



Hình 12. Demo mô hình hoạt động

3.5 Các khó khăn gặp phải và hướng phát triển trong tương lai

Trong quá trình thực hiện đồ án, em đã gặp phải nhiều khó khăn trong việc tiếp cận những kiến thức mới, lựa chọn mô hình đánh giá cũng như mô hình huấn luyện. Một số khó khăn của em khi thực hiện đồ án như là trong quá trình thu thập dữ liệu, dữ liệu bị khuyết, hay không thể vừa tạo pose vừa thu lại frame vào bộ dữ

liệu, khó khăn trong lựa chọn tham số tăng cường dữ liệu, tham số huấn luyện, đánh giá overfitting... Tuy nhiên, bằng cách tìm tòi, đọc và tham khảo thông tin, kiến thức từ nhiều nguồn cũng như thử nghiệm nhiều lần, đề án đã được hoàn thành. Dẫu cho vẫn còn một số điểm chưa được ưng ý, nhưng đây là thành quả của quá trình học tập và rèn luyện của em trong thời gian vừa qua.

Về phương hướng phát triển trong tương lai, em sẽ thực hiện nâng cấp thêm về bộ dữ liệu (số class, dữ liệu gốc đa dạng hơn) cũng như các phương pháp đánh giá, huấn luyện hiệu quả hơn. Hơn nữa, ứng dụng sẽ không chỉ dừng lại ở động tác gym mà phát triển rộng ra, phân loại, dự đoán và cảnh báo về nhiều tình huống hơn nữa. Ngoài ra, em cũng sẽ cố gắng giải quyết vấn đề còn tồn đọng hiện nay, đơn cử như việc MediaPipe hiện chỉ hỗ trợ trích xuất khung xương và keypoints cho một người trong khung hình. Em dự định sẽ tìm hiểu và sử dụng các công cụ khác như OpenPose hoặc YOLO để xử lý vấn đề này.

KẾT LUẬN

Qua quá trình thực hiện đồ án, em đã có cơ hội áp dụng kiến thức về học máy và xử lý dữ liệu hình ảnh để xây dựng một hệ thống nhận diện tư thế tập luyện trong phòng gym. Đồ án không chỉ giúp em hiểu sâu hơn về mô hình Random Forest, kỹ thuật xử lý pose từ MediaPipe, mà còn rèn luyện tư duy phân tích dữ liệu, cách đánh giá mô hình và tổ chức quy trình thực nghiệm.

Mặc dù vẫn còn nhiều hạn chế và có thể cải tiến thêm trong tương lai, em cảm thấy rất hài lòng với kết quả đạt được. Dự án đã giúp em củng cố kiến thức, đồng thời nâng cao kỹ năng lập trình và làm việc với thư viện học máy thực tế.

Em xin chân thành cảm ơn thầy Lưu Quang Trung đã tận tình hướng dẫn và tạo điều kiện để em thực hiện đồ án này một cách thuận lợi và hiệu quả.